

GROUP 10 · KNUST Computer Science Department

Automated Library Management System

DATABASE DATA DICTIONARY

Supabase PostgreSQL · v1.0 · June 2026

Aligned to SRS v1.0 — KNUST Computer Science Department

Table of Contents

Table of Contents	1
1. Document Overview	2
1.1 Database Summary	3
1.2 Out-of-Scope Entities.....	3
1.3 Key Legend.....	3
2. Custom PostgreSQL ENUM Types.....	3
3. Entity Definitions.....	4
2.1 users	5
Indexes & Constraints — users	5
2.2 books	6
Indexes & Constraints — books.....	7
2.3 borrow_records	7
Indexes & Constraints — borrow_records	8
2.4 fines	9
Indexes & Constraints — fines.....	9
4. Entity Relationships.....	10
5. Constraints Summary	11
6. Business Rules & SRS Compliance	11
7. Required Database Triggers	13
8. SRS Alignment Notes.....	14
8.1 Technology Stack.....	15
8.2 Single-Table Author / Genre Design.....	15
8.3 Borrow Limit Enforcement	15
8.4 JWT Sessions (FR-04).....	15
8.5 Fines	15
9. Document Revision History.....	15

1. Document Overview

This Data Dictionary defines the complete PostgreSQL database schema for the Automated Library Management System (ALMS), as specified in SRS v1.0. It documents every table, column, data type, constraint, and index required to implement the system.

This document is scoped strictly to the functional requirements in SRS §3 and the out-of-scope exclusions in SRS §6. Features listed as out of scope — including fine payment / billing integration, book reservation / hold queues, email / SMS notifications, and AI-based recommendations — are not represented in this schema.

The database engine is **Supabase PostgreSQL** (via Prisma ORM), consistent with the technology stack defined in SRS §5. UUID v4 primary keys are used throughout, generated by PostgreSQL's `gen_random_uuid()` function. The schema is designed to support the scalability requirements of SRS §4: up to 10,000 book records and 5,000 members.

1.1 Database Summary

Table	Row Count Estimate	Description
users	Up to 5,000	All system users: students, librarians, admins.
books	Up to 10,000	Library book catalogue with availability tracking.
borrow_records	Unbounded	Active and historical loan records.
finest	Subset of loans	Overdue fine records for reporting.

1.2 Out-of-Scope Entities

The following entities and integrations are explicitly excluded from the v1.0 schema, consistent with SRS §6. No tables are created to support them; they are noted here for traceability.

- Fine payment / billing integration — no `payment_transactions` or `invoices` tables.
- Book reservation / hold queue system — no `reservations` or `holds` table.
- Automated email or SMS notifications — no `notifications` or `message_log` table.
- AI-based book recommendations — no `recommendation` or `user_preferences` table.

1.3 Key Legend

Badge	Meaning	Notes
PK	Primary Key	Every table uses a UUID v4 generated by <code>gen_random_uuid()</code> .
FK	Foreign Key	Enforces referential integrity; ON DELETE behaviour noted per column.
UQ	Unique	Single-column unique constraint. Partial indexes used where the column is nullable.
IX	Index	Non-unique B-tree index for query performance.
CK	Check	Column-level or table-level CHECK constraint.

2. Custom PostgreSQL ENUM Types

The following ENUM types must be created in the PostgreSQL public schema before running Prisma migrations. They enforce column-level domain integrity at the database engine level.

ENUM Type Name	Values	Usage Notes / SRS Ref
user_role	'student', 'librarian', 'admin'	Assigned at registration (FR-03). Determines access level (FR-16, FR-17, FR-18). Checked on every API request via JWT claim.
loan_status	'active', 'returned', 'overdue'	'active': book is out on loan (FR-09). 'returned': return processed (FR-11). 'overdue': past due_date, set by scheduled job (FR-18). Note: 'lost' status is excluded as it has no SRS requirement.
fine_status	'unpaid', 'paid', 'waived'	Tracks fine state for FR-18 reporting. Payment processing is out of scope for v1.0 (SRS §6) but the column is included for future compatibility.

3. Entity Definitions

2.1 users

SRS References: FR-01, FR-02, FR-03, FR-04

Central authentication and role table. Every system actor (student, librarian, administrator) is a single row in this table. Role-based access control is enforced at the Express.js API layer using the role column embedded in the JWT token. Supports up to 5,000 registered members per NFR scalability requirement.

Column	Data Type	Nullable	Default	Key	References	Description & Constraints
id	UUID	NO	gen_random_uuid()	PK		Surrogate primary key. UUID v4.
full_name	VARCHAR(100)	NO				Full display name of the user. Required at registration (FR-01).
email	VARCHAR(100)	NO		UQ		Email address. Used as login credential (FR-02). Must be unique. Format validated by application layer.
password_hash	TEXT	NO				bcrypt hash of the user password (cost factor ≥ 12). Plain-text passwords are never stored. Satisfies NFR Security requirement.
role	user_role	NO	'student'			PostgreSQL ENUM: 'student' / 'librarian' / 'admin'. Assigned upon registration (FR-03). Defaults to 'student'.
is_active	BOOLEAN	NO	TRUE			Soft-delete / account suspension flag. FALSE disables login without deleting records.
created_at	TIMESTAMPTZ	NO	now()			Record creation timestamp.
updated_at	TIMESTAMPTZ	NO	now()			Last update timestamp. Maintained by trigger trg_users_updated_at.

Indexes & Constraints — users

Index / Constraint Name	Type	Columns	Purpose
users_pkey	PRIMARY KEY	id	Uniquely identifies each user.
users_email_key	UNIQUE	email	Prevents duplicate accounts. Required for FR-02 login by email.
idx_users_role	INDEX	role	Fast filtering by role for admin user management (FR-16).
idx_users_is_active	INDEX	is_active	Speeds up active-user queries at login.

Index / Constraint Name	Type	Columns	Purpose
chk_users_role_valid	CHECK	role	role IN ('student','librarian','admin').

2.2 books

SRS References: FR-05, FR-06, FR-07, FR-08

Master catalogue of all library books. Each row represents a distinct title. The available_quantity column is updated by trigger on every borrow or return event, enabling real-time availability checks without a join. Supports up to 10,000 book records per NFR scalability requirement. Author, genre, and ISBN are stored as columns per FR-07.

Column	Data Type	Nullable	Default	Key	References	Description & Constraints
id	UUID	NO	gen_random_uuid()	PK		Surrogate primary key. UUID v4.
title	VARCHAR(255)	NO				Full title of the book. Required. Supports full-text search for FR-05/FR-06.
author	VARCHAR(255)	NO				Author name(s). Required per FR-07. Multiple authors separated by semicolons.
isbn	VARCHAR(13)	YES	NULL	UQ		ISBN-10 or ISBN-13. Unique when not NULL (partial index). NULL permitted for items without an ISBN.
genre	VARCHAR(100)	YES	NULL			Genre or subject classification (e.g. 'Computer Science', 'Fiction'). Supports FR-06 filter by genre.
quantity	SMALLINT	NO	1			Total physical copies owned by the library. CHECK: quantity >= 0. Required per FR-07.
available_quantity	SMALLINT	NO	1			Copies currently available for borrowing. Updated by trigger on borrow/return (FR-12). CHECK: available_quantity >= 0 AND available_quantity <= quantity.
added_by	UUID	YES	NULL	FK	users.id	Librarian or admin who added this record (FR-08). SET NULL on user deletion.

Column	Data Type	Nullable	Default	Key	References	Description & Constraints
created_at	TIMESTAMPTZ	NO	now()			Record creation timestamp.
updated_at	TIMESTAMPTZ	NO	now()			Last update timestamp. Maintained by trigger.

Indexes & Constraints — books

Index / Constraint Name	Type	Columns	Purpose
books_pkey	PRIMARY KEY	id	Uniquely identifies each book title.
books_isbn_key	UNIQUE	isbn (partial)	Prevents duplicate ISBNs (WHERE isbn IS NOT NULL).
idx_books_title	INDEX	title	Supports title-based search and filtering (FR-06).
idx_books_author	INDEX	author	Supports author-based search and filtering (FR-06).
idx_books_genre	INDEX	genre	Supports genre-based filtering (FR-06).
idx_books_available	INDEX	available_quantity	Supports availability filter (FR-06). Filter: available_quantity > 0.
fk_books_added_by	FOREIGN KEY	added_by	References users(id) ON DELETE SET NULL.
chk_books_quantity	CHECK	quantity	quantity >= 0.
chk_books_avail_quantity	CHECK	available_quantity	available_quantity >= 0 AND available_quantity <= quantity.

2.3 borrow_records

SRS References: FR-09, FR-10, FR-11, FR-12, FR-13, FR-14, FR-15, FR-17, FR-18

Core loan tracking table. Created when a member borrows a book (FR-09). Due date is automatically set to borrow_date + 14 days (FR-10). The status column drives the member dashboard (FR-14), borrowing history (FR-15), admin borrow view (FR-17), and overdue report (FR-18). The 5-book borrow limit (FR-13) is enforced by the API before insert by counting active loans per user.

Column	Data Type	Nullable	Default	Key	References	Description & Constraints
id	UUID	NO	gen_random_uuid()	PK		Surrogate primary key.
user_id	UUID	NO		FK	users.id	The borrowing member. RESTRICT delete — cannot delete a user with loan records.
book_id	UUID	NO		FK	books.id	The book title borrowed. RESTRICT delete — cannot delete a book with loan history.
processed_by	UUID	YES	NULL	FK	users.id	Librarian or admin who processed the loan. NULL if self-service. SET NULL on user deletion.

Column	Data Type	Nullable	Default	Key	References	Description & Constraints
borrow_date	DATE	NO	CURRENT_DATE			Date the book was borrowed.
due_date	DATE	NO				Return deadline. Set by trigger: borrow_date + 14 days (FR-10). CHECK: due_date > borrow_date.
return_date	DATE	YES	NULL			Actual date the book was returned (FR-11). NULL while the book is still on loan. CHECK: return_date >= borrow_date.
status	loan_status	NO	'active'			ENUM: 'active' / 'returned' / 'overdue'. 'overdue' is set by a scheduled job when due_date < CURRENT_DATE AND status = 'active'. Used for FR-14, FR-17, FR-18.
notes	TEXT	YES	NULL			Optional librarian notes at time of borrowing or return.
created_at	TIMESTAMPTZ	NO	now()			Timestamp of loan creation.
updated_at	TIMESTAMPTZ	NO	now()			Last update timestamp (e.g. when return_date is set or status changes to overdue).

Indexes & Constraints — borrow_records

Index / Constraint Name	Type	Columns	Purpose
borrow_records_pkey	PRIMARY KEY	id	Uniquely identifies each loan record.
idx_br_user_id	INDEX	user_id	Retrieves a member's full borrowing history (FR-15).
idx_br_book_id	INDEX	book_id	Retrieves all loans for a specific book (FR-17).
idx_br_status	INDEX	status	Supports overdue loan report (FR-18): filter status = 'overdue'.
idx_br_due_date	INDEX	due_date	Used by scheduled job that transitions active loans to overdue.
idx_br_user_status	INDEX	user_id, status	Composite — powers member dashboard active-loan view (FR-14). Also used to count active loans for the 5-book limit check (FR-13).
fk_br_user	FOREIGN KEY	user_id	References users(id) ON DELETE RESTRICT.
fk_br_book	FOREIGN KEY	book_id	References books(id) ON DELETE RESTRICT.

Index / Constraint Name	Type	Columns	Purpose
fk_br_processed_by	FOREIGN KEY	processed_by	References users(id) ON DELETE SET NULL.
chk_br_due_date	CHECK	due_date	due_date > borrow_date.
chk_br_return_date	CHECK	return_date	return_date >= borrow_date (when not NULL).
chk_br_status	CHECK	status	status IN ('active','returned','overdue').

2.4 fines

SRS References: FR-18

Tracks overdue fines associated with loan records. A fine record is created automatically when a borrow_record transitions to 'overdue' status. Fine payment processing and billing integration are explicitly out of scope for v1.0 (SRS §6); this table records the fine amount for reporting and member awareness only. The status column supports future payment integration.

Column	Data Type	Nullable	Default	Key	References	Description & Constraints
id	UUID	NO	gen_random_uuid()	PK		Surrogate primary key.
borrow_id	UUID	NO		FK / UQ	borrow_records.id	The loan that generated this fine. UNIQUE — one fine record per overdue loan. RESTRICT delete.
user_id	UUID	NO		FK	users.id	The member who owes the fine. Denormalised from borrow_records for direct query. RESTRICT delete.
amount	NUMERIC(8,2)	NO				Total fine amount in GHS. Calculated at time of overdue detection. CHECK: amount > 0.
status	fine_status	NO	'unpaid'			ENUM: 'unpaid' / 'paid' / 'waived'. Payment processing is out of scope for v1.0 but the column is included for future integration.
issued_at	TIMESTAMPTZ	NO	now()			Timestamp the fine record was created.
notes	TEXT	YES	NULL			Admin or librarian notes (e.g. reason for waiver).

Indexes & Constraints — fines

Index / Constraint Name	Type	Columns	Purpose
fines_pkey	PRIMARY KEY	id	Uniquely identifies each fine.
fines_borrow_key	UNIQUE	borrow_id	Enforces one fine per overdue loan.
idx_fines_user_id	INDEX	user_id	Retrieves all fines for a member for admin overdue report (FR-18).
idx_fines_status	INDEX	status	Filters unpaid fines for reporting.
fk_fines_borrow	FOREIGN KEY	borrow_id	References borrow_records(id) ON DELETE RESTRICT.
fk_fines_user	FOREIGN KEY	user_id	References users(id) ON DELETE RESTRICT.
chk_fines_amount	CHECK	amount	amount > 0.
chk_fines_status	CHECK	status	status IN ('unpaid','paid','waived').

4. Entity Relationships

The ALMS v1.0 schema consists of four tables with the following relationships:

Relationship	From	To	Cardinality	FK Behaviour
Member borrows Books	users	borrow_records	1 : N	ON DELETE RESTRICT — cannot delete a user with loan history.
Book is borrowed	books	borrow_records	1 : N	ON DELETE RESTRICT — cannot delete a book with loan history.
Librarian processes Loan	users	borrow_records (processed_by)	1 : N	ON DELETE SET NULL — loan preserved if librarian account deleted.
Overdue Loan generates Fine	borrow_records	finest	1 : 1	ON DELETE RESTRICT — cannot delete a loan with an associated fine.
Member owes Fine	users	finest	1 : N	ON DELETE RESTRICT — fine record preserved if user account deactivated.
Librarian adds Book	users	books (added_by)	1 : N	ON DELETE SET NULL — book preserved if librarian account deleted.

5. Constraints Summary

The table below summarises all constraint types applied across the v1.0 schema.

Type	Key Examples	Purpose
PRIMARY KEY	users.id, books.id, borrow_records.id, finest.id	UUID v4 surrogate keys throughout. Prevents enumeration and supports distributed inserts via Prisma.
FOREIGN KEY	borrow_records.user_id → users, borrow_records.book_id → books, finest.borrow_id → borrow_records	Enforces referential integrity. ON DELETE behaviour defined per relationship (RESTRICT or SET NULL).
UNIQUE	users.email, books.isbn (partial), finest.borrow_id	Prevents duplicate accounts, ISBN entries, and multiple fines per loan.
NOT NULL	full_name, email, password_hash, title, author, quantity, borrow_date, due_date, amount, etc.	Ensures mandatory business data is always captured.
CHECK	available_quantity BETWEEN 0 AND quantity; due_date > borrow_date; return_date >= borrow_date; amount > 0; role/status ENUM validation	Enforces valid value ranges and logical business rules at the database engine level.
DEFAULT	gen_random_uuid(), now(), CURRENT_DATE, 'student', TRUE, 'active', 'unpaid', 1	Provides sensible automatic values, reducing application boilerplate.
ON DELETE RESTRICT	borrow_records → users/books; finest → borrow_records/users	Blocks deletion of parent rows that have dependent loan or fine records, protecting data integrity.
ON DELETE SET NULL	borrow_records.processed_by → users; books.added_by → users	Preserves the record but clears the staff reference if the staff account is removed.

6. Business Rules & SRS Compliance

The following business rules are derived directly from the SRS and are enforced by a combination of database constraints, triggers, and API-layer logic.

SRS Ref	Requirement	Enforcement Layer	Implementation
FR-01	Register with name, email, password	Database + API	users table requires full_name, email (UNIQUE), password_hash. Validation at Express.js layer.
FR-02	Authenticate via email + password	API (JWT)	API queries users by email, verifies bcrypt hash, returns JWT. FR-04: JWT stored client-side (no server token table required in v1.0).
FR-03	Assign role on registration	Database DEFAULT	users.role ENUM defaults to 'student'. Librarian/admin roles set by an existing admin via API.
FR-04	Maintain sessions via JWT	API (stateless)	JWT issued on login. No server-side session table. Token expiry enforced by the Express.js middleware.
FR-05	Searchable book catalogue	Database INDEX	idx_books_title and idx_books_author indexes support WHERE title ILIKE and author ILIKE queries via Prisma.
FR-06	Filter by title, author, genre, availability	Database INDEX	Dedicated indexes on title, author, genre, available_quantity support all four filter dimensions.
FR-07	Book record: title, author, ISBN, genre, quantity, availability	Database columns	All six attributes are columns on the books table with appropriate types and NOT NULL constraints.
FR-08	Librarians/admins add, edit, remove books	API role check	API middleware checks JWT role IN ('librarian','admin'). books.added_by tracks who added each record.
FR-09	Member borrows available book	API + Database trigger	API checks available_quantity > 0 and active loan count <= 5 before insert. Trigger decrements available_quantity.
FR-10	Due date = borrow_date + 14 days	Database trigger	trg_set_due_date sets due_date = borrow_date + INTERVAL '14 days' on INSERT into borrow_records.
FR-11	Member returns borrowed book	API + Database trigger	API sets return_date = CURRENT_DATE and status = 'returned'. Trigger increments books.available_quantity.
FR-12	Update availability on borrow/return	Database trigger	trg_update_availability decrements/increments books.available_quantity on borrow_records INSERT/UPDATE.
FR-13	Maximum 5 active borrows per member	API pre-check	Before INSERT, API counts WHERE user_id = ? AND status = 'active'. Uses idx_br_user_status composite index.
FR-14	Member views current borrows + due dates	API query	SELECT * FROM borrow_records WHERE user_id = ? AND status = 'active'. idx_br_user_status.

SRS Ref	Requirement	Enforcement Layer	Implementation
FR-15	Member views borrowing history	API query	SELECT * FROM borrow_records WHERE user_id = ? ORDER BY created_at DESC. idx_br_user_id.
FR-16	Admin views and manages all members	API role check	API middleware checks role = 'admin'. SELECT * FROM users.
FR-17	Admins view all active and past borrow records	API role check	role IN ('admin','librarian'). SELECT * FROM borrow_records with JOIN to users and books.
FR-18	Librarians and admins view overdue loans	Database + scheduled job	Scheduled Postgres cron job sets status = 'overdue' WHERE due_date < CURRENT_DATE AND status = 'active'. Admin queries filter by status = 'overdue'. idx_br_status.
NFR — Security	Passwords hashed with bcrypt	Database constraint	password_hash is NOT NULL TEXT. API uses bcrypt (cost >= 12) before storing.
NFR — Scalability	10,000 books, 5,000 members	Indexes + UUID PKs	All query-critical columns are indexed. UUID PKs avoid integer overflow. Supabase PostgreSQL handles this scale with ease.

7. Required Database Triggers

The following PostgreSQL triggers must be implemented to maintain data consistency automatically. These are defined as Prisma raw SQL migrations or directly in the PostgreSQL database.

Trigger Name	Table	Event	Action / SRS Ref
trg_set_due_date	borrow_records	BEFORE INSERT	Sets due_date = borrow_date + INTERVAL '14 days' when not supplied (FR-10).
trg_update_availability	books	AFTER UPDATE on borrow_records	Decrements books.available_quantity on borrow (status → 'active'); increments on return (status → 'returned'). Satisfies FR-12.
trg_auto_overdue_fine	borrow_records	AFTER UPDATE (status)	When status changes to 'overdue', inserts a fine record into fines with amount = days_overdue × 0.50 GHS (FR-18).
trg_users_updated_at	users	BEFORE UPDATE	Sets updated_at = now() on every row update.
trg_books_updated_at	books	BEFORE UPDATE	Sets updated_at = now() on every row update.
trg_borrow_updated_at	borrow_records	BEFORE UPDATE	Sets updated_at = now() on every row update.

Additionally, a scheduled PostgreSQL cron job (using pg_cron or a Render cron service) must run daily to execute:

```
UPDATE borrow_records SET status = 'overdue', updated_at = now()
WHERE status = 'active' AND due_date < CURRENT_DATE;
```

This satisfies FR-18 (overdue loan detection) without requiring a real-time trigger on every API call.

8. SRS Alignment Notes

This section documents specific SRS decisions that influence the schema and clarifies deviations from earlier draft data dictionaries.

8.1 Technology Stack

SRS §5 specifies MySQL + Prisma ORM as the database layer. This Data Dictionary uses **Supabase PostgreSQL** instead of MySQL. Supabase provides a fully managed, hosted PostgreSQL instance with the pgcrypto extension enabled by default, which supplies the `gen_random_uuid()` function used throughout this schema. All column types, ENUM definitions, and trigger syntax are written for standard PostgreSQL and require no Supabase-specific SQL extensions. Prisma's `schema.prisma` file must set `provider = "postgresql"` and point `datasource url` at the Supabase connection string, and must define the three ENUM types (`user_role`, `loan_status`, `fine_status`) before migration.

8.2 Single-Table Author / Genre Design

SRS FR-07 lists the required book attributes as: title, author, ISBN, genre, quantity, and availability status. The SRS does not specify separate Author, Publisher, Category, or Branch entities. Accordingly, author and genre are stored as VARCHAR columns directly on the books table — consistent with a v1.0 student project scope. A future v2.0 schema may normalise these into lookup tables.

8.3 Borrow Limit Enforcement

SRS FR-13 specifies a maximum of 5 active borrows per member. SRS §2.3 also states this limit as a system assumption. This rule is enforced at the Express.js API layer (pre-insert count check using `idx_br_user_status`) rather than by a database CHECK constraint, because CHECK constraints cannot perform subquery row counts in standard PostgreSQL without a custom function. The composite index `idx_br_user_status (user_id, status)` ensures this check is always fast.

8.4 JWT Sessions (FR-04)

SRS FR-04 states the system shall maintain user sessions using JWT tokens. The SRS does not require server-side token storage or refresh token rotation. Accordingly, no `refresh_tokens` table is included in the v1.0 schema. JWTs are issued on login by the Express.js backend and verified stateless on each request. A blacklist / refresh token table may be added in a future version.

8.5 Fines

SRS FR-18 requires librarians and admins to view overdue loans. The fines table is included to record the fine amount generated when a loan becomes overdue, supporting the overdue loans view. However, SRS §6 explicitly excludes online fine payment and billing integration. The `fines.status` column is included for future use but payment processing endpoints are not part of v1.0.

9. Document Revision History

Version	Date	Author	Changes
v1.0	June 2026	ALMS Project Team	Initial release. 4-table schema strictly aligned to SRS v1.0. Users, books, borrow_records, fines. Supabase PostgreSQL with Prisma ORM. 3 ENUM types. 6 triggers.